DAFFODIL INTERNATIONAL UNIVERSITY

Department of Software Engineering | Faculty of Science & Information Technology

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Statute-Grounded Bangla Legal Assistant

A Status-Aware Legal Retrieval-Augmented Generation (RAG) System for the Bangladesh Labour Act, 2006 (incorporating 2013, 2018 & 2026 Amendments)

Only for Course Teacher — Evaluation Rubric (Semester: Fall 2026)					
Assessment Criteria	**Needs Improvement (25%)**	**Developing (50%)**	**Sufficient (75%)**	**Above Average (100%)**	**Total Mark (25)**
Problem Understanding & Analysis (7)					
Implementation & Architecture (8)					
Task Efficiency & SRS Alignment (10)					
				Total Obtained Mark:	
Comments:					

Course & Instructor Information	Submitted By — Development Team (Batch 42, Section B1)
Course Name: Software Engineering Design and Capstone Project **Course Code:** SE331 **Course Teacher:** Rahat Uddin Azad **Designation:** Lecturer, Dept. of Software Engineering **Semester:** Fall 2026 | **Batch:** 42 | **Section:** 42-B1 **Submission Date:** 15/10/2026 (Updated Baseline: Nov 2026) **Repository:** `Statute-Grounded-Bangla-Legal-Assistant`	**1. Ashikur Rahman Sazim** Student ID: `241-35-034` | Role: Backend Core, Auth & Database Lead **2. Prarthona Biswas Puja** Student ID: `241-35-187` | Role: Legal Chunker & Vector DB Engineer **3. Sabiha Nazir (Ani)** Student ID: `241-35-279` | Role: Data Pipeline, RAG Integration & UI Lead

1. Introduction

This Software Requirements Specification (SRS) defines the functional, non-functional, and architectural requirements for the **Statute-Grounded Bangla Legal Assistant**. The software is a secure, Bangla-first legal information web application engineered for workers, employers, law students, and general citizens of Bangladesh. Focusing deeply on the **Bangladesh Labour Act, 2006 (Act No. XLII of 2006)** along with its subsequent statutory amendments (2013, 2018, and 2026), the platform enables authenticated users to ask legal questions in everyday Bangla or search specific section numbers (`ধারা`) and receive accurate, plain-Bangla explanations strictly grounded in official statutory text.

1.1 Problem Statement

When workers or citizens in Bangladesh face workplace disputes—such as unpaid wages, overtime calculation, maternity benefits, unlawful termination, or trade union rights—finding reliable legal provisions is difficult. The official Bangladesh Laws portal (`bdlaws.minlaw.gov.bd`) only supports rigid keyword matching and displays lengthy formal statutes where footnoted amendments and repealed clauses are mixed at the bottom of the webpage. Furthermore, general-purpose AI chatbots (such as standard ChatGPT) are not grounded in updated Bangladeshi statute snapshots: they frequently hallucinate section numbers, confuse Indian labour laws with Bangladeshi law, or cite repealed sections (such as Section 184 or Dock Worker Sections 253–262) as active law. Standard character-based text splitters also break legal sub-clauses apart, corrupting the legal meaning.

1.2 Purpose & Core Objectives

- **Compulsory Secure Authentication:** Provide a complete User Registration and Login module with strict input validation and cryptographic password hashing (`bcrypt`) stored in a relational database (`SQLite`), ensuring personalized consultation logs and zero hardcoded credentials.
- **Status-Aware Statutory Chunking (Core Requirement 1):** Automatically scrape, audit, and segment the Bangladesh Labour Act, 2006 into **375 self-contained statutory sections**, cross-referencing 1,177 amendment footnote lines to classify every section as **Active (183 sections)**, **Amended (181 sections)**, or **Repealed (11 sections)** with exact gazette provenance.
- **Hybrid Semantic & Metadata-Filtered Retrieval (Core Requirement 2):** Index all 375 enriched chunks into a persistent **ChromaDB** vector database using multilingual Bangla embeddings, allowing users to retrieve provisions by semantic meaning, exact section number, or legal status filter.
- **Citation-Backed AI Consultation & Audit Logging (Core Requirement 3):** Generate plain-Bangla answers strictly from retrieved sections, explicitly warning users when a queried section is repealed or amended, verifying all citations, and saving every user query, citation, and feedback record into the relational database.

1.3 Scope & AI Boundaries

The system operates as a Bangla-first web application backed by a modular Python data pipeline, FastAPI REST endpoints, ChromaDB vector storage, and SQLite relational persistence. The system provides verified legal information and statutory citations only; it does not provide formal legal representation, predict court verdicts, or fabricate provisions outside the indexed corpus. When a query falls outside the Bangladesh Labour Act, 2006 or falls below the retrieval confidence threshold, the system safely refuses to guess and directs the user to qualified legal aid.

2. Design and Implementation Constraints

2.1 Technology Stack & Engineering Justification

System Layer	Selected Technology	Technical Justification
Data Scraping & Regex Chunker	Python 3.11+, `Requests`, `BeautifulSoup4`, Custom Regex	Handles UTF-8 Bangla Unicode normalization, Bengali digits (`০-৯`), and dual danda punctuation (`।`, `॥`) to split 4,389 raw lines into 375 exact statutory sections without mid-clause truncation.
Vector Database (RAG Store)	`ChromaDB` (Persistent Local Client) + `Sentence-Transformers`	Stores dense multilingual embeddings (`paraphrase-multilingual-MiniLM-L12-v2`) alongside rich metadata (`section_number`, `legal_status`, `chapter`, `amendment_provenance`) for instant semantic and metadata filtering.
Relational Database & Auth	`SQLite3` + `Passlib/Bcrypt` + `Pydantic`	Provides zero-configuration ACID relational storage for the compulsory Authentication module (`users` table) and persistent consultation history (`query_logs` and `feedback` tables).
Backend API & LLM Engine	`FastAPI` + Bangla-capable LLM API (Groq / Gemini / Llama-3)	Delivers asynchronous REST endpoints, automatic OpenAPI/Swagger documentation, strict Pydantic input validation, and grounded prompt execution with citation verification.
Frontend Interface	`Streamlit` / Responsive HTML5 Web UI	Provides a clean Bangla user interface featuring User Registration/Login forms, interactive legal chat, status filter badges (`Active`, `Amended`, `Repealed`), expandable source cards, and user history.

2.2 Statutory Corpus Metrics (Verified Implementation Baseline)

Unlike generic prototypes that rely on unverified text snippets, our system's data pipeline has been audited and executed against the official Bangladesh Labour Act, 2006 (`act-952.html` from `bdlaws.minlaw.gov.bd`). The resulting corpus metrics are locked as follows:

- **Raw Dataset Volume:** 4,389 non-empty UTF-8 lines (3,212 primary statutory body lines + 1,177 historical amendment and gazette footnote lines).
- **Total Processed Statutory Chunks: 375 Sections** (`backend/data/processed/labour_act_chunks.json`).
- **Status Distribution: 183 Active (`সক্রিয়`)** sections (unchanged since 2006), **181 Amended (`সংশোধিত`)** sections (modified via 2013, 2018, or 2026 amendments), and **11 Repealed (`বিলুপ্ত`)** sections (Section 184 repealed by the 2026 Amendment Act + Sections 253–262 Dock Workers Board repealed by the 2013 Amendment Act and superseded by Section 263A).

3. User Classes and Characteristics

User Class	Characteristics	System Permissions & Access
Registered Citizen / Worker	Primary end-user seeking clarity on wages, leave, termination, or workplace rights in simple Bangla.	Register account, log in securely, ask Bangla legal questions, filter sections by status, view exact statutory citations and amendment history, view personal query history, and submit feedback.
Law Student / Legal Aid Researcher	Requires rapid section lookup, exact statutory wording, and historical tracking of amended or repealed clauses.	Full authenticated access to hybrid section search, status-aware filtering (`Active` / `Amended` / `Repealed`), provenance inspection, and consultation logs.
System Administrator (Project Team)	Technical maintainers managing the scraping pipeline, chunking rules, and database indices.	Run automated crawler (`01_scraper.py`), execute status-aware chunker (`02_legal_chunker.py`), rebuild ChromaDB index (`03_vector_store.py`), and audit SQLite logs.

4. Requirement Elicitation & Domain Validation

Requirements were gathered and refined through two primary elicitation channels prior to implementation:

1. **Online User Survey (Google Forms, n = 9 respondents, Oct 9–12, 2026):** Conducted among university and college students. Key findings showed that **78%** needed legal information in Bangladesh, **67%** found locating the exact section (`ধারা`) difficult on government portals, **78%** rated displaying the exact section number and original text as highly important (4–5 out of 5), and **100%** supported having an explicit reliability/status indicator to prevent outdated or hallucinated legal answers.

2. **In-Person Domain Expert Consultation (Oct 10, 2026):** Conducted with **Samir Bin Shibbir**, Lecturer, Department of Law, Daffodil International University. The faculty expert emphasized three critical rules: (a) answers must cite the exact section (ধারা) rather than vague summaries, (b) users must be informed of statutory amendments and repealed provisions with clear source attribution, and (c) the system must refuse to answer when a query is unsupported by the statute and must never dispense prescriptive legal advice.

5. Functional Requirements (Aligned with Sprint Review)

To ensure complete traceability between this SRS and our codebase implementation, the functional requirements are organized into the **Compulsory Authentication Module (FR-01)** and the **Three Core Functional Modules (FR-02, FR-03, FR-04)**, each backed by direct database interaction.

Req. ID	Module & Feature Name	Detailed Technical Requirement	Database Interaction
FR-01.1 (Compulsory)	**User Registration & Input Validation**	The system shall allow new users to register with full name, valid email address, and password. It shall validate email format, check for duplicate accounts, and enforce password length rules (minimum 6 characters).	**SQLite (`users` table):** `SELECT` to check existing email; `INSERT` new user record with timestamp.
FR-01.2 (Compulsory)	**Secure Password Hashing & Login**	The system shall never store plaintext or hardcoded passwords. Passwords must be hashed using `bcrypt` / PBKDF2-SHA256 before storage. Upon login, credentials shall be verified against the stored hash to issue an authenticated session/JWT token.	**SQLite (`users` table):** `SELECT password_hash` by email and verify cryptographic digest.
FR-02.1 (Core 1)	**Automated Statutory Scraping & Raw Audit**	The system shall scrape the Bangladesh Labour Act, 2006 from the official BDLaws portal, verify UTF-8 integrity, and separate the 3,212 main statutory lines from the 1,177 amendment footnote lines.	**File & Audit Store:** Persists `labour_act_raw.txt` and generates `audit_report.txt` .
FR-02.2 (Core 1)	**Status-Aware Legal Chunker & Vector Indexing**	The system shall parse all 375 sections (ধারা), distinguish whole-section repeals from sub-clause amendments, recover repealed sections from footnotes/ranges (Sections 184, 253–262), tag each as `Active (183)` , `Amended (181)` , or `Repealed (11)` , and index embeddings into ChromaDB.	**ChromaDB Vector Store (`labour_act_collection`):** Upserts 375 vector embeddings + metadata fields.

Req. ID	Module & Feature Name	Detailed Technical Requirement	Database Interaction
FR-03.1 (Core 2)	**Hybrid Semantic & Exact Section Retrieval**	The system shall accept user queries in Bangla (or section numbers like `ধারা ১`, `ধারা ১৮৪`, `ধারা ২৫৩`), combine exact section-number matching with dense multilingual vector search (top-k = 5), and support filtering by legal status.	**ChromaDB Vector Store:** Executes `collection.query()` and `collection.get(where=...)` against indexed vectors.
FR-03.2 (Core 2)	**Evidence Confidence & Low-Score Refusal**	The system shall compute a retrieval confidence status (`Supported`, `Partly Supported`, or `Not Supported`). If similarity falls below threshold or the topic is outside the Labour Act, the system shall refuse to guess.	**ChromaDB & SQLite:** Reads distance scores from ChromaDB and logs refusal status in `query_logs`.
FR-04.1 (Core 3)	**Citation-Backed Bangla AI Consultation**	The system shall synthesize a plain-Bangla answer using only the retrieved sections, displaying the exact section number, chapter, legal status badge (`সক্রিয়` / `সংশোধিত` / `বিলুপ্ত`), amendment provenance, original statutory text, and legal disclaimer.	**SQLite (`query_logs` table):** `INSERT` authenticated `user_id`, query, cited sections, AI answer, and status.
FR-04.2 (Core 3)	**User Consultation History & Feedback Logging**	Authenticated users shall be able to view their past legal queries and citations fetched from the database, and submit helpful/not-helpful ratings with optional comments for system evaluation.	**SQLite (`query_logs` & `feedback` tables):** `SELECT` user history by `user_id`; `INSERT` feedback rating.

6. Non-Functional & AI Quality Requirements

- **NFR-01 (Zero Hallucination & Citation Faithfulness):** 100% of sections cited in the generated response must exist in the retrieved ChromaDB context. Repealed sections (such as Section 184 or Sections 253–262) must never be presented as active law; the response must explicitly state the repeal Act and year.
- **NFR-02 (Cryptographic Security):** User passwords must be salted and hashed before writing to `SQLite`. No hardcoded credentials or API keys may exist in source files; sensitive keys are loaded via `.env` variables.
- **NFR-03 (Bangla Unicode & Encoding Integrity):** All file I/O, regex patterns, and database transactions must strictly enforce `UTF-8` encoding without Mojibake corruption across Windows and Linux environments.
- **NFR-04 (Response Latency & Fallback):** Vector search and grounded answer generation shall complete within 5–10 seconds on a standard connection. If an external LLM API key is absent or unreachable, a deterministic Statute-Grounded Fallback Engine shall still return the exact retrieved sections, status badges, and provenance from ChromaDB without system crash.

7. System Architecture, Use Cases & Database Schema

7.1 End-to-End Architectural Pipeline

> **Offline Ingestion Pipeline:** Official BDLaws HTML (`01_scraper.py`) → Raw UTF-8 Corpus (4,389 lines) → Raw Data Audit (`audit_raw_act.py`) → Status-Aware Legal Chunker (`02_legal_chunker.py` → 375 JSON Chunks: 183 Active, 181 Amended, 11 Repealed) → Multilingual Embedding & ChromaDB Indexing (`03_vector_store.py`).
>
> **Online Authenticated RAG Flow:** User Registers/Logs In (`auth.py` + SQLite `users`) → Submits Bangla Query / Section Filter → Hybrid Search in ChromaDB (Exact Section Match + Semantic Vector Search) → Status & Provenance Verification → Grounded Bangla Answer Generation + Citation Card Display → Persistent Audit Log (`query_logs` & `feedback` in SQLite).

7.2 Core Use Case Specifications

Use Case ID	Use Case Name & Actor	Main Success Flow & Database Verification
UC-01 (FR-01.1, FR-01.2)	**Register & Authenticate User** Actor: Citizen / Student	1. User enters name, email, and password. 2. System validates inputs and hashes password via `bcrypt` . 3. Account is saved in SQLite `users` table. 4. User logs in; system verifies hash and grants access to the consultation workspace.
UC-02 (FR-02.1, FR-02.2)	**Build Status-Aware Vector Knowledge Base** Actor: System Admin	1. Admin runs `02_legal_chunker.py` to segment 375 sections and resolve footnote/range repeals. 2. Admin runs `03_vector_store.py` . 3. System embeds all 375 chunks and persists vectors + status metadata in ChromaDB.
UC-03 (FR-03.1, FR-03.2)	**Search Statute by Query or Status Filter** Actor: Authenticated User	1. User searches a topic (e.g., maternity leave, overtime) or specific section (e.g., `ধারা ১` , `ধারা ১৮৪` , `ধারা ২৫৩`) and optionally selects a status filter (`Active` / `Amended` / `Repealed`). 2. System queries ChromaDB and returns top-k matching sections with exact metadata and confidence status.
UC-04 (FR-04.1, FR-04.2)	**Receive Grounded Consultation & Save History** Actor: Authenticated User	1. System generates a plain-Bangla explanation grounded strictly in retrieved sections, highlighting amendment/repeal provenance. 2. Query, cited section IDs, and answer are logged in SQLite `query_logs` . 3. User views past history and submits feedback stored in SQLite `feedback` .

7.3 Dual-Database Schema (SQLite Relational DB + ChromaDB Vector Store)

Database & Table / Collection	Primary & Foreign Keys / IDs	Stored Attributes & Purpose
SQLite: `users`	`user_id` (INTEGER PK AUTOINCREMENT) `email` (TEXT UNIQUE NOT NULL)	`full_name`, `password_hash`, `role`, `created_at`. Handles compulsory user registration and secure login verification.
SQLite: `query_logs`	`log_id` (INTEGER PK AUTOINCREMENT) `user_id` (INTEGER FK → `users.user_id`)	`query_text`, `cited_sections`, `legal_status_summary`, `ai_response`, `confidence_status`, `created_at`. Tracks consultation audit history per user.
SQLite: `feedback`	`feedback_id` (INTEGER PK AUTOINCREMENT) `log_id` (INTEGER FK → `query_logs.log_id`)	`user_id` (FK), `rating` (`Helpful` / `Not Helpful`), `comment`, `created_at`. Stores user evaluation of AI responses.
ChromaDB: `labour_act_sections`	Document ID: `sec_{section_number}` (e.g., `sec_১`, `sec_১৮৪`, `sec_২৫৩`)	**Embedding:** 384-d dense vector. **Metadata:** `act_name`, `chapter`, `section_number`, `section_title`, `legal_status`, `amendment_provenance`.

8. Team Member Responsibility & Git Ownership Matrix

Each team member owns a distinct technical module, dedicated Git feature branch, and database interaction layer to ensure transparent individual evaluation during the Sprint Review:

Team Member & ID	Git Branch & GitHub Identity	Owned Modules, Files & Assessment Demonstration Scope
Ashikur Rahman Sazim ID: `241-35-034`	Branch: `feat/backend-core` User: `rahmansazim`	**Compulsory Auth Module (FR-01) & Backend Core Architecture:** • Designed SQLite relational schema (`users`, `query_logs`, `feedback`) in `backend/database.py`. • Implemented User Registration, Login validation, and cryptographic password hashing (`bcrypt`) in `backend/auth.py`. • Built FastAPI core service endpoints and session security in `backend/main.py`.

Team Member & ID	Git Branch & GitHub Identity	Owned Modules, Files & Assessment Demonstration Scope
Prarthona Biswas Puja ID: `241-35-187`	Branch: `feat/legal-chunker` User: `Prarthona Biswas`	**Core Requirement 1 (FR-02) & Core Requirement 2 (FR-03) — Legal Chunker & ChromaDB Vector Store:** • Built raw corpus auditor (`backend/scripts/audit_raw_act.py`) and resolved UTF-8/Mojibake anomalies. • Engineered `backend/scripts/02_legal_chunker.py` to segment 375 sections (`183 Active` , `181 Amended` , `11 Repealed`), fixing the sub-clause vs. whole-section repeal bug and recovering repealed sections (184, 253–262). • Implemented ChromaDB vector indexing and metadata search in `backend/scripts/03_vector_store.py` .
Sabiha Nazir (Ani) ID: `241-35-279`	Branch: `feat/data-pipeline` User: `Ani`	**Data Acquisition Pipeline & Core Requirement 3 (FR-04) — RAG Consultation UI & Audit Logging:** • Implemented automated BDLaws web crawler (`backend/scripts/01_scraper.py`) to acquire the 4,389-line raw Labour Act dataset. • Built the citation-grounded RAG consultation engine (`backend/rag_engine.py`) and interactive Bangla Web UI (`app.py`) integrating Login, Status Filtering, Citation Cards, and SQLite Consultation History/Feedback logging.